THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

ASSIGNMENT 7

Aadhya Maheshwari—1024150212

1.

Class `polygon` contains data members width and height and public method `set_value()` to assign values. Classes `Rectangle` and `Triangle` are inherited from polygon class. Both classes contain public method `calculate_area()` to calculate the area.
Use base class pointer to access derived class object and show the area calculated.

main.cpp | Share | Run | Output | Clear

```cpp
#include <iostream>
using namespace std;

class Polygon {
protected:
    int width, height;
public:
    void set_value(int w, int h) {
        width = w;
        height = h;
    }
    virtual void calculate_area() = 0;
};

class Rectangle : public Polygon {
public:
    void calculate_area() {
        cout << "Area of Rectangle: " << width *
            height << endl;
    }
};

class Triangle : public Polygon {
public:
    void calculate_area() {
        cout << "Area of Triangle: " << (width *
            height) / 2 << endl;
    }
};
```

```
Area of Rectangle: 50
Area of Triangle: 25


=== Code Execution Successful ===
```

```
int main() {
    Polygon *p;
    Rectangle r;
    Triangle t;
    p = &r;
    p->set_value(10, 5);
    p->calculate_area();
    p = &t;
    p->set_value(10, 5);
    p->calculate_area();
    return 0;}
```

2.

Write a program to create a class `shape` with functions `area` and `display` to find area and display the name of the shape and other essential components.
Create derived classes `circle`, `rectangle`, and `triangle` each having overridden functions `area` and `display`.
Display the area of all shapes.

main.cpp | Share | Run | Output | Clear

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void area() = 0;
    virtual void display() = 0;
};

class Circle : public Shape {
    float r;
public:
    Circle(float x) { r = x; }

    void area() {
        cout << "Area: " << 3.14 * r * r << endl;
    }

    void display() {
        cout << "Shape: Circle" << endl;
    }
};

class Rectangle : public Shape {
    int l, b;
public:
    Rectangle(int x, int y) { l = x; b = y; }

    void area() {
        cout << "Area: " << l * b << endl;
    }
```

```
Shape: Circle
Area: 78.5
Shape: Rectangle
Area: 24
Shape: Triangle
Area: 12


=== Code Execution Successful ===
```

```
32
33      void display() {
34          cout << "Shape: Rectangle" << endl;
35      }
36  };
37
38  class Triangle : public Shape {
39      int b, h;
40  public:
41      Triangle(int x, int y) { b = x; h = y; }
42
43      void area() {
44          cout << "Area: " << (b * h) / 2 << endl;
45      }
46
47      void display() {
48          cout << "Shape: Triangle" << endl;
49      }
50  };
51
52  int main() {
53      Shape *s;
54
55      Circle c(5);
56      Rectangle r(4,6);
57      Triangle t(3,8);
58
59      s = &c; s->display(); s->area();
60      s = &r; s->display(); s->area();
61      s = &t; s->display(); s->area();
62
63      return 0;
64  }
```

3.

Write a C++ program to compute area of:

- **Right angle triangle**
- **Equilateral triangle**
- **Isosceles triangle**
  using function overloading.

main.cpp | Share | Run | Output | Clear

```cpp
#include <iostream>
#include <cmath>
using namespace std;

float area(float b, float h) {
    return 0.5 * b * h;
}

float area(float side) {
    return (sqrt(3) / 4) * side * side;
}

float area(float a, float b, float c) {
    float s = (a + b + c) / 2;
    return sqrt(s * (s - a) * (s - b) * (s - c));
}

int main() {
    cout << "Right Triangle Area: " << area(5,6) <<
        endl;
    cout << "Equilateral Triangle Area: " << area(4)
        << endl;
    cout << "Isosceles Triangle Area: " << area(5,5,6)
        << endl;

    return 0;
}
```

```
Right Triangle Area: 15
Equilateral Triangle Area: 6.9282
Isosceles Triangle Area: 12


=== Code Execution Successful ===
```

4.

Write a program with `Student` as abstract class and create derived classes:

- **Engineering**
- **Medicine**
- **Science**

Create objects of derived classes and access them using an array of base class pointers.

main.cpp | Share | Run | Output | Clear

```cpp
#include <iostream>
using namespace std;

class Student {
public:
    virtual void show() = 0;
};

class Engineering : public Student {
public:
    void show() {
        cout << "Engineering Student" << endl;
    }
};
class Medicine : public Student {
public:
    void show() {
        cout << "Medicine Student" << endl;
    }
};
class Science : public Student {
public:
    void show() {
        cout << "Science Student" << endl;
    }
};
int main() {
    Student* s[3];
    Engineering e;
    Medicine m;
    Science sc;
    s[0] = &e;
    s[1] = &m;
    s[2] = &sc;

    for(int i = 0; i < 3; i++) {
        s[i]->show();
    }
    return 0;}
```

```
Engineering Student
Medicine Student
Science Student


=== Code Execution Successful ===
```

5.

Create a class `Time` with private variables (h, m, s).

Overload `+` operator to add two time objects.

int main(){

Time t1(5,15,34), t2(9,53,58), t3;

t3 = t1 + t2;

t3.show();

}

```cpp
#include <iostream>
using namespace std;

class Time {
    int h, m, s;

public:
    Time(int a=0, int b=0, int c=0) {
        h = a;
        m = b;
        s = c;
    }

    Time operator + (Time t) {
        Time temp;

        temp.s = s + t.s;
        temp.m = m + t.m + temp.s / 60;
        temp.s = temp.s % 60;

        temp.h = h + t.h + temp.m / 60;
        temp.m = temp.m % 60;

        return temp;
    }

    void show() {
        cout << h << ":" << m << ":" << s << endl;
    }
};

int main() {
    Time t1(5,15,34), t2(9,53,58), t3;

    t3 = t1 + t2;
    t3.show();

    return 0;
}
```

```
15:9:32


=== Code Execution Successful ===
```

6.

Define a class `STRING` and overload:

- **`==` to compare two strings**
- **`+` for concatenation**

main.cpp Share Run Output

```cpp
#include <iostream>
#include <string>
using namespace std;

class STRING {
    string str;

public:
    STRING(string s="") {
        str = s;
    }

    bool operator == (STRING s) {
        return str == s.str;
    }

    STRING operator + (STRING s) {
        return STRING(str + s.str);
    }

    void show() {
        cout << str << endl;
    }
};

int main() {
    STRING s1("Hello "), s2("World"), s3;

    s3 = s1 + s2;
    s3.show();

    if(s1 == s2)
        cout << "Strings are Equal\n";
    else
        cout << "Strings are Not Equal\n";

    return 0;
}
```

```
Hello World
Strings are Not Equal


=== Code Execution Successful ===
```

7.

Write a C++ program to create a class `matrix` and overload `*` operator using friend function to multiply two matrices.

```cpp
#include <iostream>
using namespace std;

class Matrix {
    int a[2][2];

public:
    void input() {
        for(int i=0;i<2;i++)
            for(int j=0;j<2;j++)
                cin >> a[i][j];
    }

    void display() {
        for(int i=0;i<2;i++) {
            for(int j=0;j<2;j++)
                cout << a[i][j] << " ";
            cout << endl;
        }
    }

    friend Matrix operator*(Matrix, Matrix);
};

Matrix operator*(Matrix m1, Matrix m2) {
    Matrix temp;

    for(int i=0;i<2;i++)
        for(int j=0;j<2;j++) {
            temp.a[i][j] = 0;
            for(int k=0;k<2;k++)
                temp.a[i][j] += m1.a[i][k] * m2
                    .a[k][j];
        }
    return temp;
}

int main() {
    Matrix m1, m2, m3;

    cout << "Enter Matrix 1:\n";
    m1.input();

    cout << "Enter Matrix 2:\n";
    m2.input();

    m3 = m1 * m2;

    cout << "Result:\n";
    m3.display();

    return 0;
}
```

```
Enter Matrix 1:
1 2 3 4
Enter Matrix 2:
5 6 7 8
Result:
19 22
43 50


=== Code Execution Successful ===
```

8.

Overload [ ] to check array index out of bounds at runtime.

main.cpp | Share | Run | Output | Clear

```cpp
#include <iostream>
using namespace std;

class Array {
    int a[5];

public:
    int& operator[](int i) {
        if(i < 0 || i >= 5) {
            cout << "Index Out of Bounds\n";
            exit(0);
        }
        return a[i];
    }
};

int main() {
    Array arr;

    arr[0] = 10;
    cout << arr[0] << endl;

    return 0;
}
```

```
10


=== Code Execution Successful ===
```

9.

Overload ( ) to input arbitrary number of input arguments for an object.

main.cpp | Share | Run | Output | Clear

```cpp
#include <iostream>
using namespace std;

class Demo {
public:
    void operator()(int a, int b, int c) {
        cout << "Values: " << a << " " << b << " " <<
            c << endl;
    }
};

int main() {
    Demo d;
    d(1,2,3);

    return 0;
}
```

```
Values: 1 2 3


=== Code Execution Successful ===
```

10.

Write a program in C++ to overload:

- **Input operator >>**
- **Output operator <<**

main.cpp | Share | Run | Output | Clear

```cpp
#include <iostream>
using namespace std;

class Student {
    int roll;

public:
    friend istream& operator>>(istream &in, Student &s) {
        cout << "Enter Roll: ";
        in >> s.roll;
        return in;
    }

    friend ostream& operator<<(ostream &out, Student &s) {
        out << "Roll No: " << s.roll;
        return out;
    }
};

int main() {
    Student s;

    cin >> s;
    cout << s;

    return 0;
}
```

```
Enter Roll: 1024150212
Roll No: 1024150212

=== Code Execution Successful ===
```

11.

Write a program to convert basic data type (float) to user-defined data type (object) using constructor.

main.cpp Share Run Output Clear

```cpp
#include <iostream>
using namespace std;

class Test {
    float x;

public:
    Test(float a) {
        x = a;
    }

    void show() {
        cout << x << endl;
    }
};

int main() {
    float a = 5.5;

    Test t = a;
    t.show();

    return 0;
}
```

```
5.5


=== Code Execution Successful ===
```

12.

Write a program to convert user-defined data type (UDT) to basic data type (float) using conversion operator.

```cpp
#include <iostream>
using namespace std;

class Test {
    float x;
public:
    Test(float a) {
        x = a;
    }
    operator float() {
        return x;
    }};
int main() {
    Test t(5.5);
    float a = t;
    cout << a << endl;
    return 0;}
```

```
5.5


=== Code Execution Successful ===
```

13.
Convert one UDT to another UDT (example: Polar to Cartesian).

Polar p(10,5);
Cartesian c = p;
c.show();

main.cpp

```cpp
#include <iostream>
#include <cmath>
using namespace std;

class Polar {
    float r, theta;

public:
    Polar(float a, float b) {
        r = a;
        theta = b;
    }

    float getR() { return r; }
    float getTheta() { return theta; }
};

class Cartesian {
    float x, y;

public:
    Cartesian(Polar p) {
        x = p.getR() * cos(p.getTheta());
        y = p.getR() * sin(p.getTheta());
    }

    void show() {
        cout << "x = " << x << ", y = " << y <<
            endl;
    }
};

int main() {
    Polar p(10,5);
    Cartesian c = p;

    c.show();

    return 0;
}
```

Output

```
x = 2.83662, y = -9.58924


=== Code Execution Successful ===
```